

Theraptly LMS — System Analysis Report

Audit date: 2026-07-05 · **Branch:** `dev` @ `4f47527` · **Codebase:** `lms2` v0.1.0 **Scale:** ~75,700 LOC TypeScript/TSX · 674 commits since 2026-02-26 · 25 Prisma migrations · 50 API route handlers · 31 server-action modules · 68 test files (553 unit + 20 e2e cases)

This is the full technical tier. For a one-page verdict see [EXECUTIVE-SUMMARY.md](#) ; for the flat finding list see [FINDINGS-REGISTER.md](#) . Every claim is anchored to `file:line` .

1. What the system is

Theraptly is a multi-tenant, AI-powered Learning Management System for **healthcare compliance training**. An organization (tenant) uploads policy documents; a Vertex AI / Gemini pipeline turns them into courses, slides, and quizzes; workers are enrolled, complete training, take quizzes, and receive certificates; admins track compliance and export auditor packs. It scans uploaded documents for **PHI (Protected Health Information)**, which places it squarely in HIPAA scope, and it bills organizations through Stripe.

Stack: Next.js 16.2.6 (App Router, React 19), NextAuth v5 (JWT, dual admin/worker realms), Prisma 7 + `@prisma/adapters-pg` on PostgreSQL 16 + pgvector, BullMQ + Redis for jobs, MinIO + Google Cloud Storage for files, Vertex AI (`gemini-2.5-flash-lite` , `text-embedding-004`), Stripe, nodemailer (Zoho SMTP), Tailwind v4 + shadcn/ui. Deployed on a single VM via Docker Compose behind nginx + a Cloudflare Tunnel.

Overall verdict

The codebase is **more mature than its 0.1.0 version implies** and has clearly survived at least one adversarial security pass — the twelve findings recorded in `docs/hack-check.md` are all genuinely remediated, and the security fundamentals (parameterized SQL, DOMPurify, signed-URL proxies, Stripe signature verification, secure cookies, CSPRNG tokens) are largely correct. Engineering hygiene is good: zero stray `console.*`, a structured logger, `strict` TypeScript, and 553 unit cases over the pure-logic libraries.

The risk is concentrated in **three structural gaps**, each of which is individually a launch-blocker for a HIPAA product at scale:

- Compliance instrumentation is missing.** There is no audit log (F-001), no backups (F-004), no encryption at rest (F-025), and PHI is transmitted to Google *before* verification and stored raw (F-002, F-003). None of these are hard to fix, but none can be skipped for HIPAA/SOC 2.
- The architecture is a single-process monolith that assumes one instance.** Background workers and cron run *inside* the web process and only start when a human loads `/system` (F-005, F-015); document parsing and multi-minute AI calls block request handlers (F-016); nothing is cached (F-046). This is why the app needs a 4 GB heap and cannot scale horizontally — and it is exactly what the requested frontend/backend split must undo.
- Tenant isolation is 100% application-enforced with no backstop.** The schema gives `Course` and `Enrollment` no `organizationId` (F-007); there's no RLS and no scoping middleware;

correctness depends on every developer remembering a `where` clause. Two such clauses were already missed — the new IDORs in `staff.ts` (F-009, F-010).

The rest of this report walks each subsystem, states its architecture, and lists findings by severity. The [rebuild documentation](#) turns the structural findings into an actionable target design.

2. Data model & persistence

Architecture. Prisma 7 multi-file schema (12 `.prisma` files, `snake_case @@map / @map` throughout) on PostgreSQL 16 with the `pgvector` extension. The client is generated to `generated/prisma` via the new `prisma-client` generator; a second legacy `prisma-client-js` generator exists only for a raw Node transcode worker. A `PrismaPg` driver adapter wraps a `pg` pool. Tenancy is single-level, rooted at `Organization`, and **mostly transitive**: only `users`, `invites`, `course_categories`, `course_assignments`, `org_course_offerings`, `subscriptions`, and `invoices` carry `organization_id` directly. Everything else reaches its tenant by joining through `User.organizationId`.

Findings.

- **F-006 (Critical)** — the `pgvector manual_chunks.embedding vector(768)` column exists only via raw SQL and is absent from the Prisma model, so the next model-driven migration will try to `DROP` it.
- **F-007 (Critical)** — no DB-level tenant isolation; `Course / Enrollment` have no `organizationId`; no RLS. This is the schema root of the IDOR class of bugs.
- **F-026 (High)** — the Prisma "singleton" in `src/lib/prisma.ts` writes `globalForPrisma.prisma` but never reads it back, so HMR leaks pools and there are two client entry points (`@/db` and `@/lib/prisma`); `PrismaPg` gets no pool sizing.
- **F-027 (High)** — `lessons.course_id` and `questions.quiz_id` (the primary access paths) are unindexed FKs; the RAG KNN query has no `ivfflat / hnsw` index and sequential-scans.
- **F-032 (High)** — `QuizAttempt @@unique([enrollmentId, quizId])` allows only one attempt row, silently overwriting history and contradicting `allowedAttempts` and the retry states.
- **F-052, F-053 (Medium)** — org-scoped category slug is globally unique; no unique on `(user_id, course_id)` enrollments.

Migration health. 25 migrations, a squashed `0_init` baseline, and several hand-typed round-number timestamps (`20260504100000`, `20260518020000`, ...) betraying manually authored folders. One migration (`20260518015115_add_compliance_document`) is a committed **0-byte no-op**. The big `20260622135211_rename_tables_columns` (typo in the folder name) is 368 lines of manual `RENAME` to `snake_case` — well-crafted but irreversible in practice. The vector-column migration (F-006) is the one standing drift hazard.

Rebuild-critical facts are captured in `../rebuild/04-DATA-STORAGE-SPEC.md`: all 14 enums, the six `raw*Json` AI-output columns on `Course`, the JSON column shapes, the `String[]` defaults, the shared-PK 1:1 `Profile*User`, the PK-less `verification_tokens`, and the full `onDelete` cascade policy — all of which must be recreated exactly to reuse the existing database.

3. Authentication, authorization & multi-tenancy

Architecture. NextAuth v5 with a JWT strategy and **two independent realms** built from one factory (`src/lib/create-auth-instance.ts`): admin (`admin.session-token`) and worker (`worker.session-token`), which can be logged in simultaneously. The only difference between them is cookie name and the `allowedRole` enforced at three points (`authorize` , the jwt callback, and the proxy). Credentials use bcrypt; Microsoft Entra ID OAuth is optional; MFA is TOTP (`otpauth`) with AES-256-GCM-encrypted secrets, email-OTP fallback, and bcrypt-hashed recovery codes; challenge state and per-session MFA verification live in Redis (fail-closed). Middleware is `src/proxy.ts` (Next 16 renamed `middleware` → `proxy`); it decodes the realm cookie, enforces role, forced-password-reset, MFA step-up, and worker-onboarding gating — **but only for page routes**. A separate signed-cookie "system admin" realm gates `/api/system/**` with a single shared password.

The core weakness is that authorization is enforced per-handler, opt-in, and is not uniformly applied. There is no central authz layer, no Prisma tenant-scoping extension, and the proxy's matcher excludes every business API route and every server action.

Findings.

- **F-009, F-010 (High)** — two cross-tenant IDORs: `getStaffDetails` and `getEnrollmentQuizResult` in `staff.ts` do a session check but no org scoping, leaking another tenant's worker PII, enrollment history, and quiz **answer keys**. Every *mutation* in the same file scopes correctly; these two *reads* were missed — the exact "auth checked, object-authz missed" pattern from the prior report.
- **F-012 (High)** — MFA is only enforced in the proxy (page routes); a password-only session (`mfaVerified=false`) can call any API route or action directly.
- **F-013 (High)** — business API routes bypass the proxy entirely; security is opt-in per handler.
- **F-033 (Medium)** — login throttle lives in the server action, not `authorize()` , so the raw NextAuth credential callback is unthrottled (per-account logout also absent).
- **F-034 (Medium)** — `course.ts` / `lesson.ts` mutators check only `session.user.id` , not `role` ; a worker session can create courses.
- **F-035, F-036, F-037 (Medium)** — `AUTH_SECRET` / `NEXTAUTH_SECRET` split-brain in the proxy; jwt callback fail-open on DB error; no throttle on password-reset request or invite-accept.
- **F-056-F-059 (Low)** — weak system-admin auth; email in reset redirect; mixed bcrypt costs; reset doesn't invalidate sessions.

Multi-tenancy is the throughline: because the schema provides no isolation (F-007) and there's no scoping primitive, tenant safety is a discipline, not a guarantee. The rebuild must centralize a default-deny `requireSameOrg(resource)` guard. See [../rebuild/07-SECURITY-COMPLIANCE-SPEC.md](#) .

4. API surface & server actions

Architecture. The server-side contract is split across **two mechanisms**: 50 REST route handlers under `src/app/api/**` and **31 'use server' action modules** (~120 functions) that client components call as RPC — plus **26 server-component pages that query Prisma directly at render time**. Roughly half the data flow bypasses the REST layer entirely. Error shapes are inconsistent (`{error}` vs `{success:false,error}` vs zod `{error,details}` ; actions `throw new Error('Unauthorized')` with no status). Validation is zod on a handful of routes (quiz, enrollment

progress, invite) and hand-rolled `if (!field)` casts everywhere else. Rate limiting exists only on MFA-send and resend-verification.

Findings.

- **F-014 (High)** — Stripe webhook returns 200 on handler error (Stripe never retries → permanent desync) and has no event-id idempotency.
- **F-015 (High)** — workers boot from request handlers/page renders; the single biggest coupling to undo for the split.
- **F-016 (High)** — heavy work (document parsing, AI generation, quiz-grading AI call, report generation) runs synchronously in request handlers.
- **F-018 (High)** — no rate limiting on AI endpoints.
- **F-048 (Medium)** — document-proxy route returns raw internal error text to the client.

The complete endpoint inventory — every method, path, purpose, observed auth guard, validation, and side effect — is reproduced as the **backend API contract** in `../rebuild/03-BACKEND-SPEC.md`, because that inventory *is* the surface the new backend must expose.

5. AI pipeline, document processing & jobs

Architecture (v4.6). Course generation: upload → text extraction (`pdf-parse` / `mammoth`) → optional RAG retrieval (pgvector over an indexed `StandardManual`) → a staged Vertex chain (Article → Slides → Quiz → Judge → Regen-flagged), each stage a `callVertexAI` REST call with per-stage temperature/token config. The pipeline runs **in-process via Next's `after()`**, writing status to a DB `Job` row that the wizard polls. PHI scanning is a separate Vertex call that samples the first 15k characters and **fails closed** (any error/malformed response → `hasPHI:true`). Jobs use two unrelated mechanisms: the DB `Job` table (client-facing status) and BullMQ (`manual-indexer` , `video-transcode` , `video-sweep` , `reminder-sweep` , `auditor-export`) — all workers running inside the web process.

Findings.

- **F-002 (Critical)** — the course-wizard file-upload path never calls `scanText` ; document text goes straight to Vertex with no PHI gate. This is a PHI-scan bypass on the *primary* course-creation flow.
- **F-003 (Critical)** — PHI is sent to Google to detect PHI, and matched values are stored raw in `phi_reports.detected_entities` .
- **F-005 (Critical)** — workers/cron only start on a `/system` page-load; reminders and escalations silently stop after any restart.
- **F-016, F-017 (High)** — blocking AI in `after()` /handlers; no upload size cap; whole file buffered in memory (the source of the 4 GB heap).
- **F-018 (High)** — no AI rate limiting / cost controls.
- **F-049, F-050, F-051 (Medium)** — `BLOCK_NONE` safety + unescaped prompt-injection surface; `String.replace` substitution corrupts on `$` -sequences; degraded generations auto-publish with no review gate.
- **F-066 (Low)** — embeddings call has no timeout.

Positive: PHI reaches only the BAA-eligible Vertex endpoint (OAuth service account); the non-BAA consumer Gemini SDK is a dependency but unused. Preserve this boundary. Full pipeline + worker rebuild spec: `../rebuild/05-AI-PIPELINE-WORKERS-SPEC.md` .

6. Frontend

Architecture. Next.js App Router with route groups for public/marketing, `(auth)`, admin `dashboard/(main)` and `(wizard)`, `worker`, onboarding, and a super-admin `system` area. The CSS-Module → Tailwind/shadcn migration is **genuinely complete** (0 `.module.css`). The critical structural fact for the split: the frontend reaches directly into backend concerns everywhere — 26 pages query Prisma at SSR, 31 server-action modules embed Prisma + storage + queue + Stripe + email + AI, and there is **no client-side HTTP abstraction** at all.

Findings.

- **F-061 (Low)** — two divergent job-polling implementations; the robust hook has one consumer while the auditor path reinvented a weaker poller.
- **F-062 (Low)** — `<Toaster>` mounted only in the admin layout; `toast()` no-ops everywhere else.
- **F-063 (Low)** — 84% of components are client components; recharts/react-pdf/xlsx statically imported (bundle bloat).
- **F-064 (Low)** — no `error.tsx` boundaries, one `loading.tsx`, despite server components doing live DB I/O.
- **F-065 (Low)** — design drift: 78 files with raw hex in `className`, 26 with inline styles, two competing token sources.

The retained-frontend requirements per area (what each page needs from the new backend) are in `../rebuild/02-FRONTEND-SPEC.md`.

7. Billing, email, certificates & cross-cutting

Billing (Stripe). Three plans, in-place price-swap on upgrade (THER-001), signature-verified webhook with a canonical-row guard (THER-010). But: **F-014** (200-on-error, no idempotency), **F-022** (seat limits never enforced post-checkout; `staffCount` is free text), **F-023** (hCaptcha absent), **F-040** (portal pause never expires), **F-041** (null-able default export).

Email & notifications. 19 nodemailer templates; an idempotent reminder ladder + throttled nudges swept by a daily cron. But **F-020** (failed emails marked "sent" via a dedup row written *before* send, never retried) and **F-021** (no bounce/delivery tracking) make compliance reminders unreliable — the worst place for silent failure in this product.

Audit & reporting. The "auditor pack" export pipeline is solid and tenant-isolated on download, but the `AuditorPack` model is **orphaned/never written**, and there is **no audit log at all (F-001)** — PHI-adjacent exports and certificate downloads are unlogged (**F-054** retention gap compounds this).

Certificates. Server-side pdfkit generation, idempotent issuance, public QR verification. But **F-038** (public page leaks email via `fullName || email`), **F-039** (`score || 100` zeroes-out bug), and no revocation path.

Rate limiting & headers. Redis sliding-window on auth paths only; **F-018** (no AI limits), **F-019** (no security headers anywhere), **F-024** (per-process fail-open fallback).

Logging. Structured logger, zero stray `console.*`, `maskEmail` — but **F-067** (stdout only, no shipping/retention/correlation IDs; `serializeError` spreads all error props) and inconsistent masking (`email.ts:80-82` logs a raw address and a tokenized invite link).

Subsystem rebuild details are distributed across the backend, security/compliance, and infrastructure specs.

8. Infrastructure & operations

Architecture. Single VM, Docker Compose per environment (dev/staging/prod), app behind nginx + a Cloudflare Tunnel. Postgres/Redis/MinIO run as containers on host bind mounts with no published ports; GCS is primary storage with MinIO fallback. Two deploy mechanisms coexist: current Docker/GHCR GitHub Actions and legacy PM2 shell scripts (**F-029**).

Findings.

- **F-004 (Critical)** — no backups of Postgres, MinIO, or Redis; all on one disk.
- **F-005 (Critical)** — cron/workers gated behind a human page-load (see §5).
- **F-008 (Critical)** — Gemini key baked into the browser build via `NEXT_PUBLIC_`.
- **F-025 (High)** — no encryption at rest; document plaintext in the DB.
- **F-024 (High)** — per-instance rate-limit fallback.
- **F-030 (High)** — no CI security gates; e2e not run in CI; migrations run on every container start *and* in a deploy script (double-execution risk).
- **F-042, F-043, F-044 (Medium)** — no boot-time env validation; tunnel bypasses nginx (and the prod hostname doesn't match nginx's `server_name`); floating `:latest` image tags.
- **F-060 (Low)** — sensitive info (dev DB password, GCP SA email, test creds, defect descriptions) committed under `.claude/agent-memory/*`.

Target-state infrastructure (web / API / worker as separate deployables, private network for stateful services, backups, observability) is specified in `../rebuild/06-INFRASTRUCTURE-SPEC.md`.

9. Testing & engineering quality

Positives. 553 unit cases with strong coverage of the `lib/` pure logic (reminders, storage, PHI scanner, AI client, audit reports); a 5-job CI (lint/format/typecheck/test/build); `strict: true`; zero genuine TODO/FIXME debt; ~25 `any`; zero stray `console.*`.

Gaps.

- **F-031 (High)** — quiz scoring (the certification gate) and multi-tenancy isolation have **zero** automated tests.
- **F-030 (High)** — billing route coverage is a fraction of the surface; e2e never runs in CI; the heavy pre-commit hook (full `test` + `build`) pushes devs toward `--no-verify`.
- **F-068 (Low)** — god files (`course.ts` 1,337; `email.ts` 1,148; `learn/[id]/page.tsx` 1,118) cluster in the untested, high-churn areas.

Known issues from `qa-reports/`: the FINAL QA report rates Billing ☐ critical and Audit x; all 15 THER tickets are marked fixed on `fix/qa-report-001`, but several carry "needs live e2e to confirm"

caveats.

10. Compliance posture (HIPAA + SOC 2 Type II)

A full control matrix is in the HIPAA/SOC 2 analysis feeding `../rebuild/07-SECURITY-COMPLIANCE-SPEC.md`. The headline gaps, ranked:

Control	Status	Finding
Audit controls (§164.312(b))	Missing	F-001
PHI minimization / third-party transmission	Partial, leaking	F-002, F-003
Data backup plan (§164.308(a)(7))	Missing	F-004
Encryption at rest (§164.312(a)(2)(iv))	Missing	F-025
Transmission security	Partial (Cloudflare edge; internal HTTP)	F-025, F-043
Availability / monitoring / IR (SOC 2 A1.2, CC7)	Missing	F-004, F-030
Access control / RBAC	Partial (2 roles, MFA opt-in & API-bypassable)	F-012, F-034
Change management (SOC 2 CC8.1)	Partial (CI exists, no security gates)	F-030
PII in logs	Partial (violations exist)	F-067

Subprocessors that need a BAA: Google Cloud (Vertex AI + GCS — highest risk, receives PHI), Cloudflare (TLS terminates at its edge; Free/Pro plans are **not** BAA-eligible — verify the plan), and the SMTP provider if any message can carry PHI. Stripe and Microsoft Entra receive PII only (no BAA required / covered by DPA). No analytics/telemetry SDKs are present.

11. Prioritized remediation roadmap

Ordered by risk-reduction per unit effort. IDs reference the register.

Phase 0 — Stop the bleeding (days, before any scale or new customer):

1. Fix the two IDORs (F-009, F-010) — small, high-impact, breaks tenant isolation today.
2. Add the PHI gate to the wizard path (F-002) — one function call closes a PHI leak.
3. Remove/rotate `NEXT_PUBLIC_GEMINI_API_KEY` (F-008); move secrets & defect notes out of VCS (F-060).
4. Stripe webhook: return 5xx on retryable error + event-id dedup (F-014).
5. Add security headers (F-019) and AI-endpoint rate limits (F-018).
6. Protect the pgvector column before the next migration (F-006).

Phase 1 — Compliance floor (weeks, gates a HIPAA claim): 7. Append-only `AuditLog` written at a central boundary (F-001). 8. Automated encrypted backups + tested restore (F-004). 9. Encryption at rest; drop/encrypt `DocumentVersion.content`; store PHI types not values (F-025, F-003). 10. Move workers/cron to an always-on process (F-005) — also the first step of the split. 11. Enforce MFA on API/actions via a shared guard; centralize `requireSameOrg` (F-012, F-013, F-007). 12. Reminder-email delivery tracking + retry (F-020, F-021).

Phase 2 — Scale & structure (the frontend/backend split): 13. Extract the backend + a separate worker service; move all heavy work off the request path (F-015, F-016, F-017). 14. Add caching, pagination, media via CDN, connection pooling (F-046, F-028, F-047, F-026, F-027). 15. Single deploy mechanism, boot-time env validation, observability + alerting (F-029, F-042, F-030). 16. Test the untested: quiz scoring, cross-org isolation, billing mutations; run e2e in CI (F-031, F-030).

Phase 3 — Hardening & cleanup: the remaining Medium/Low items, dependency upgrades (F-011, F-055), design-system token adoption (F-065), god-file decomposition (F-068), stale-doc correction (F-069).

The split in Phase 2 is not merely a scaling exercise — it is the cleanest way to draw the compliance boundary (PHI, secrets, and audit logging behind one backend the frontend can't reach directly), which is why the rebuild documentation treats it as the organizing principle.